

Click-to-Source 3D

From 0.1.4 to 0.2.0: one command, an AI agent panel, and plain Three.js with nothing tagged by hand

2 October 2026 · 0.1.4 released · github.com/pun1th01/click-to-source-3d

At a glance

Version	What it brings	Status
0.1.4	Eight bug fixes: instances resolve however they were placed, Save edits the right value, nothing runs in production, monorepos work.	Released
0.1.5	One command and no wiring for React Three Fiber. The plugin injects the inspector; Alt+Shift+C turns on inspect mode; editable values are detected automatically.	Next
0.1.6	The Agent panel: a small window in the inspector that shows what your AI assistant is doing in the scene, plus new MCP tools.	Planned
0.2.0	Plain Three.js with nothing tagged by hand: the line that made a mesh, the calls that led there, and the helpers they used, such as a noise function. First version supporting both Three.js and R3F.	Planned

How to read this document

- **Parts 1–2** explain where the project stands and which package does what.
- **Part 3** is the plan for 0.1.5, the one-command setup. It is the next thing to build.
- **Parts 4–5** explain MCP from the ground up, then the Agent panel built on it.
- **Part 6** is the plan for plain Three.js in 0.2.0.
- **Part 7** is a menu of optional features. Tick the ones you want; nothing there is committed.
- Parts 8–10 cover order, decisions and risks. A glossary of terms is at the end.

PART 1

Where things stand

0.1.4 is released and published. It fixed eight bugs; the tool works well once it is wired in. The problem now is how much wiring it takes.

What 0.1.4 fixed

- Instances placed with the standard three.js loop, `setMatrixAt(i, dummy.matrix)`, now resolve. The old capture probe caught 0 of 500.
- Save edits the value that draws the mesh, never a copy inside hand-written metadata.
- Loaded models keep their own `userData` (for example glTF extras from Blender).
- Components wrapped in `memo` or `forwardRef` are named; `<animated.mesh>` is tagged.
- The write endpoint only accepts single-value edits, applied to the file as it is on disk.
- The React components render nothing in production; monorepo packages can be edited.

What still hurts: about ten setup steps

To use the tool today, a developer has to do all of this by hand:

#	Step	Removed in
1	Install two packages	0.1.5 (one command)
2	Add the plugin to <code>vite.config</code> with three options	0.1.5 (the command does it)
3	Write a click handler that resolves and stores the selection	0.1.5 (injected inspector)
4	Mount <code><SelectionHighlight /></code> inside the Canvas	0.1.5
5	Mount <code><GenerationTrace /></code> outside the Canvas	0.1.5
6	Mount <code><ClickToSourceBridge /></code> for agent tools	0.1.5
7	Add <code>onPointerMissed</code> to the Canvas	0.1.5
8	Hand-write <code>sourceRef</code> metadata, with hard-coded line numbers, to get editable values	0.1.5 (auto-detected values)
9	Hide the tools from production builds	0.1.4 (done) and 0.1.5
10	Configure the MCP server with two environment variables	0.1.5 (found automatically)

PART 2

The packages: what does the job

The project is split into five npm packages. That is normal for a tool with a build-time half and an in-browser half, but users should not have to think about it. After 0.1.5, they install exactly one.

Package	What it does today (0.1.4)	After 0.1.5
vite-plugin	The build-time and dev-server half. Tags every mesh with its file and line, serves the read/edit endpoints, and relays the AI agent's questions to the page.	The main package. It also injects the inspector, so it is the only one users install.
overlay	The in-page half: the panel, the outline, and the click handling you wire up yourself.	Legacy. Keeps working for people who wired it by hand; removed later.
core	The logic that answers "where did this object come from". Installed automatically.	Unchanged; still internal.
shared	Types and constants both halves agree on. Installed automatically.	Unchanged; still internal.
mcp	Optional. Lets AI coding assistants use the tool. Explained in Part 4.	Optional; set up by <code>init</code> if you say yes.

In short

Today the work is split between `vite-plugin` and `overlay`. From 0.1.5, `@click-to-source-3d/vite-plugin` does everything, and `overlay` becomes legacy.

PART 3

0.1.5: one command, no wiring

Goal: a React Three Fiber developer runs one command and the tool works, with no changes to their app code.

1 Run one command <code>npx click-to-source-3d init</code>	2 Start your app <code>npm run dev</code>	3 Turn on inspect mode Press Alt+Shift+C , or the button on the canvas	4 Click any mesh See its file, line and editable values
---	--	--	--

3.1 The `init` command

A new, tiny package named `click-to-source-3d` (the name is free on npm). `npx` downloads and runs it once; it is never added to the project. It:

1. Finds the project's `package.json` and works out the package manager (npm, pnpm, yarn or bun) from the lockfile.
2. Checks the project uses Vite and React Three Fiber. Without R3F it explains that plain Three.js arrives in 0.2.0 and stops.
3. Installs `@click-to-source-3d/vite-plugin` as a dev dependency.
4. Adds `clickToSource()` to `vite.config`. Running it twice changes nothing. If the config is too unusual to edit safely, it prints the two lines to paste instead.

- 5. Asks one question: Set up AI-agent tools (MCP)? (y/N). The default is No. Flags `--mcp` and `--no-mcp` skip the question.
- 6. Prints the next step: `run npm run dev` and press `Alt+Shift+C`.

3.2 The injected inspector

In dev only, the plugin adds a small script to the page before your app loads. Nothing is added to production builds, so there is nothing to hide.

- **Finding the scene.** three.js announces every scene and renderer it creates to a built-in devtools hook. The inspector listens to that hook, so it finds the scene without any code from you. For R3F it also asks R3F's store for the camera, so it can redraw scenes that only render on demand.
- **Plain DOM, no React.** The panel is built with plain browser APIs inside an isolated shadow root, so your app's CSS cannot break it and it does not care which React version you use. This also means the same inspector works for plain Three.js in 0.2.0.

3.3 Inspect mode: Alt+Shift+C

You do	What happens
Press Alt+Shift+C , or click the small button on the canvas	Inspect mode turns on. Press it again to turn it off.
Move the mouse over the scene	The mesh under the cursor is outlined, with a tooltip like <code>World.tsx:41</code> .
Click a mesh	It stays selected and the panel opens. The click does not reach your app's own handlers.
Drag	The camera still orbits as normal, so you can look around while inspecting.
Press Esc	Leaves inspect mode.
Type in a text field	The shortcut is ignored, so it never fires by accident.

The shortcut is configurable for apps that already use `Alt+Shift+C`, and the mode is remembered across reloads.

3.4 The highlight

The outline is drawn on top after your frame renders, as a separate pass. It never touches your scene graph or takes over your render loop, so it works alongside post-processing effects and keeps antialiasing on. For an instanced mesh it outlines only the clicked instance, not every instance.

3.5 The panel

- File, function and line, with an **Open in editor** button that jumps straight there (it uses the same mechanism Vite's own error overlay uses).
- The editable values detected for the mesh (3.6), each with a Save button.
- Mesh details: geometry, vertex count, material and colour, position.
- For an instance: its index and transform.
- When nothing is known about an object, it says so and names the object type and its parents, instead of showing nothing.

3.6 Editable values, with nothing tagged by hand

Today, a value only becomes editable if you hand-write `sourceRef` metadata with the right line number. In 0.1.5 the build step finds them itself. For every element it records each literal value and its exact position in

the file:

```
<mesh position=[[-2.2, 0.7, 6]]>           position x, y, z → editable
  <boxGeometry args=[[1.2, BOX_HEIGHT]] /> 1.2 → editable; BOX_HEIGHT → its "const" line
  <meshStandardMaterial color="#c2643c" /> color → editable
</mesh>
```

- A constant declared in the same file, like `BOX_HEIGHT`, is followed to its declaration and edited there.
- Anything computed, such as `noise(x, z) * 8`, is shown read-only as the code that produces it.
- Each edit names the exact position and the value it expects to find. If the file changed in the meantime, the edit is refused rather than landing on the wrong thing.
- Hand-written `sourceRef` still works as an optional override, but is never required.

3.7 Under the hood

- **The bridge** (how the AI agent asks the page questions) moves onto Vite's own websocket, which is already open. No extra connection per tab, so it can be on by default.
- **The MCP server finds the dev server by itself.** The plugin writes a small file saying which address and folder it is serving; the MCP server reads it. No environment variables.
- **No re-optimize reload on first run**, and it works under pnpm's strict layout. Both are checked on a fresh app before release.

3.8 What changes for existing users

Nothing breaks. If the old React components are still mounted, the injected inspector notices and stays out of the way, so nobody gets two panels. The README's manual setup moves to an "Advanced" section.

3.9 How we will know it works

- Unit tests for each new piece, and plugin tests against a real Vite server, as today.
- A new end-to-end browser test on the example app: press `Alt+Shift+C`, click the box, check the panel, edit a value, confirm the file changed.
- A manual run on a brand-new Vite + R3F app, under both npm and pnpm, with zero app-code edits.

PART 4

MCP, explained from the ground up

You do not need to know MCP to use Click-to-Source. This part explains what it is, what our MCP server lets an AI assistant do, and how it fits together.

4.1 What MCP is

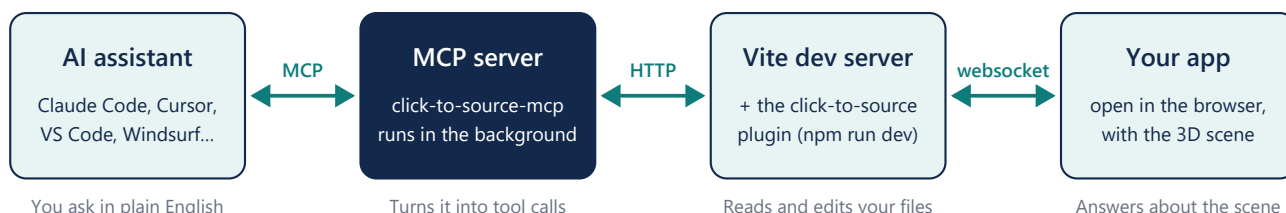
MCP stands for **Model Context Protocol**. It is an open standard that lets an AI coding assistant use tools provided by other programs. Think of it as a USB port for AI assistants: any assistant that supports MCP can plug into any MCP server and use its tools, without either side knowing about the other in advance.

Assistants that support it include Claude Code, Claude Desktop, Cursor, VS Code with Copilot, and Windsurf. On its own, an assistant can read and edit your files. With our MCP server plugged in, it can also **ask your running 3D scene questions**, such as "what is at this point on screen?" or "where in the code did this tree come from?"

Why that matters

An assistant reading your code sees what the code says. Our tools tell it what the code actually built. In the dogfooding project, two declarations in the source turned into eight instanced meshes at runtime, with counts chosen by a random seed. No amount of reading the source would reveal that.

4.2 How the pieces connect



File tools stop at the dev server. Scene tools go one step further, to the page in your browser.

- **You and the assistant.** You ask in plain English: make the box taller.
- **The MCP server** is a small program the assistant starts in the background. It has no window. It turns the assistant's requests into calls to the dev server.
- **The dev server** (Vite, with our plugin) reads and edits your source files, with the same safety rules as the panel.
- **Your app in the browser** answers questions about the live scene. This is why scene tools need the page open.

4.3 The tools the assistant gets

Tool	What the assistant can do with it	Needs the app open?
<code>list_scene_provenance</code>	List every tagged object currently in the scene, with where it came from.	Yes
<code>resolve_at_point</code>	Ask what is at a point on screen, and get its file, function and line.	Yes
<code>get_instance_provenance</code>	Get one object, or one instance in an instanced mesh, by its address.	Yes
<code>get_source</code>	Read a source file, or a window of lines around one line.	No
<code>edit_parameter</code>	Change one value in the source. Vite then hot-reloads the scene.	No
<code>list_files</code>	See which source files exist.	No
<code>list_provenance</code>	Find hand-written metadata in the source.	No
<code>search_by_generator</code>	Find code by function name or argument name.	No

4.4 An example conversation

You: "The box in the front-left is too short. Make it twice as tall."

1. The assistant calls `list_scene_provenance` and finds a box mesh made in Scene, `src/main.tsx`, with an editable value `height`.

2. It calls `get_source` around that line and sees `const BOX_HEIGHT = 1.4`.
3. It calls `edit_parameter` to change `BOX_HEIGHT` to `2.8`.
4. Vite hot-reloads. The box on your screen is now twice as tall, and the assistant tells you which line it changed.

This works today: the 0.1.3 release notes describe an assistant finding the tree generator and editing the box height this way, without reading files directly.

4.5 Setting it up

	Today (0.1.4)	From 0.1.5
Install	Install <code>@click-to-source-3d/mcp</code> yourself.	<code>init</code> asks "Set up AI-agent tools?" and installs it on yes.
Register with the assistant	Write a JSON entry by hand, with <code>CTS_DEV_SERVER</code> and <code>CTS_PROJECT_ROOT</code> .	<code>init</code> writes <code>.mcp.json</code> for you. No environment variables.
Scene tools	Also need <code>bridge: true</code> and the bridge component.	On by default. Nothing to mount.

4.6 What it cannot do yet

- **It cannot see the screen.** `resolve_at_point` takes coordinates, but the assistant has no picture to choose them from. Optional feature 5 in Part 7 fixes this.
- **It needs the dev server running,** and scene tools need the app open in a visible tab (browsers pause background tabs).
- **Instance positions are read-only.** They come from a random seed, so there is no single number in the source to change.

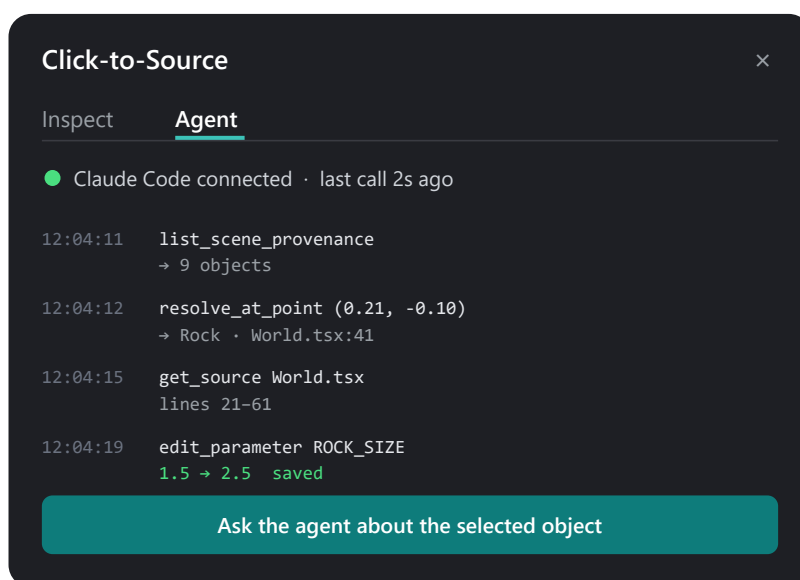
PART 5

0.1.6: the Agent panel

Your idea of a small MCP window. It makes the assistant's work in your scene visible, and lets you point it at things.

5.1 Why it lives in the browser

The MCP server has no window; it is a background process the assistant runs. But every request the assistant makes passes through the dev server, which can pass a copy to the page over Vite's websocket. So the natural home for the UI is a second tab in the inspector panel, next to the one you already use.



Mock-up of the Agent tab. Exact layout to be decided while building it.

5.2 What it shows

- **Connection status:** whether an assistant is connected, and when it last made a call.
- **A live activity log:** every tool call in plain words, with its result. Edits are marked, so you can see exactly which values were changed.
- **What the assistant is looking at:** objects it asks about are highlighted in the scene in their own colour, separate from your selection.
- **Click a log entry** to select that object or open that line in your editor.

5.3 The other direction: you point, it acts

A button, **Ask the agent about the selected object**, plus a new tool `get_selection` that returns whatever you have selected. You click a mesh and type "make this one smaller", and the assistant knows exactly which one.

5.4 New MCP tools that come with it

Tool	What it does
<code>get_selection</code>	Returns the object you have selected in the inspector: file, line, editable values and transform.
<code>highlight_object</code>	Lets the assistant point at something on your screen, for example "this is the tree I mean".

Depends on

The websocket bridge from 0.1.5, which is what lets the dev server tell the page about each call. That is why the panel is planned for 0.1.6, right after it.

PART 6

0.2.0: plain Three.js, nothing tagged

The original vision: click any mesh in a plain Three.js app and see the line that made it and how it was generated, without writing a single tag.

6.1 What you will see

This is real output from a working prototype, run on an ordinary untagged Three.js scene: a terrain shaped by an fbm noise function, and some rocks. It is what the panel would show when you click the terrain:

```
Mesh      created at world.js:13:19 in createTerrain
trace:    buildWorld(scene=<Scene>, seed=42)   world.js:27
          > createTerrain(size=200, seed=42)   world.js:4
geometry: PlaneGeometry at world.js:5   { width: 200, height: 200, widthSegments: 64, ... }
material: MeshStandardMaterial at world.js:12   color #88aa55
uses:     fbm(x, y, seed, octaves)   noise.js:6
          hash(x, y, seed)   noise.js:1
literals: 64 @5:56, 0.05 @8:38, 4 @8:70, 8 @8:75, 0x88aa55 @12:60
```

Reading it top to bottom: the exact line that created the mesh; the chain of calls that led there with the actual values passed in; the geometry and material settings; the helper functions used, including the noise function; and every number you might want to tweak, with its position, ready to edit.

6.2 How it works

1. **Mark every creation.** The build step finds each new `THREE.Mesh(...)`, new `THREE.PlaneGeometry(...)` and so on, and records its file, line and column.
2. **Record the calls that led there.** Functions that create Three.js objects get a light wrapper that remembers the values they were called with. Helper functions like noise are not wrapped, so tight loops cost nothing. In the prototype, building the world took the same time with and without it.
3. **Note what each generator uses.** The build step reads which functions each generator calls (that is how `fbm` → `hash` shows up) and which literal numbers it contains.
4. **Keep it out of your objects.** All of this lives in a lookup table in the page, not in `userData`, so exporting or saving a scene is unaffected.

The inspector from 0.1.5 already finds plain Three.js renderers, so `Alt+Shift+C` works the same way. `init` stops requiring `R3F`.

6.3 Gaps to close before release

Case	Why it is hard	Plan
Callers in other files	The wrapper is decided one file at a time.	Scan the whole project once at startup.
async generators	The recorded chain breaks across <code>await</code> .	Fall back to the creation line, or use <code>async</code> context.
Objects built inside libraries	<code>GLTFLoader</code> , <code>three/addons</code> : their code is not transformed.	Credit your <code>load()</code> call; show the <code>glTF</code> node path.
Your own classes, and <code>.clone()</code>	<code>class Tree extends THREE.Group</code> has no new <code>THREE.X</code> of its own.	Mark new <code>Tree()</code> and <code>.clone()</code> calls too.
<code>WebGPURenderer</code>	May not use the devtools hook.	Detect its render call instead.

0.2.0 is the first version supporting both

It ships the plain Three.js support, keeps R3F working through the same inspector, and removes the things 0.1.4 deprecated: `captureInstances` and `core/probe`.

PART 7

Optional features: pick what you want

Ideas that fit the vision, each with an honest size estimate. None are committed. Tick the box for the ones you want, and they get scheduled into a release.

#	Feature	What you get	Size	My take	Add?
1	Transform gizmo	Drag, rotate or scale a mesh directly in the scene, and the numbers in your source update to match.	Large	Strong: the most visible feature a 3D tool can have. Needs 0.1.5's exact-position edits.	☐
2	Live tweak sliders	Drag a slider on any numeric value to preview it live in the scene; Save writes it to the file.	Medium	Strong: much faster than typing and saving.	☐
3	Where it is used	Show not only where a mesh is declared, but where its component is used: <code>Tree.tsx:12</code> ← <code><Tree></code> in <code>Forest.tsx:30</code> .	Med–Large	Strong, but needs research into React's dev-only owner data first.	☐
4	Scene outliner	A searchable tree of every tagged object in the panel. Click one to select it.	Medium	Good for big scenes where objects are hard to click.	☐
5	Agent screenshot tool	An MCP tool that lets the assistant see the canvas, so it can choose what to ask about.	Small–Med	Strong for MCP users: fixes the biggest gap in 4.6.	☐
6	MCP Apps	Show the Agent panel inside the AI chat itself, for assistants that support MCP Apps.	Medium	Later. Depends on how widely assistants support it.	☐
7	Performance per object	Triangles, draw calls, material and texture sizes for the selected mesh: "why is this slow?"	Small–Med	Useful and cheap; three.js already exposes most of it.	☐
8	Webpack and Next.js	Support beyond Vite, so Next.js R3F projects can use the tool.	Large	Big reach: many R3F apps use Next.js.	☐
9	Change detection	After an edit or a regeneration, show which objects changed. Also fixes the known address-staleness limit.	Medium	Nice to have.	☐
10	Copy as bug report	One click copies the file, line, values, transform and a screenshot as Markdown, for an issue or a chat.	Small	Quick win.	☐
11	Pin and compare	Pin two objects side by side to compare their values.	Small	Optional polish.	☐

If you only pick three

1 (transform gizmo), 2 (live sliders) and 5 (agent screenshot). The first two make the inspector feel like an editor rather than a viewer; the third makes the MCP tools genuinely usable.

PART 8

Order and versions

Version	What ships	Builds on
0.1.4	Bug fixes (released).	—
0.1.5	init command; injected inspector with Alt+Shift+C; overlay-pass highlight; auto-detected editable values; Open in editor; bridge on Vite's websocket; MCP finds the dev server itself; end-to-end browser tests.	0.1.4
0.1.6	Agent panel; get_selection and highlight_object; any small optional features you pick.	0.1.5's websocket bridge
0.2.0	Plain Three.js with nothing tagged; both frameworks through one inspector; deprecated options removed; larger optional features you pick.	0.1.5's inspector

PART 9

Decisions made so far

Topic	Decision
Inspect shortcut	Alt+Shift+C toggles inspect mode, plus a button on the canvas. Esc exits. Configurable.
Setup command	npx click-to-source-3d init, published as a tiny separate package so setup-only code stays out of projects.
MCP during setup	Optional. init asks once, defaulting to No; --mcp and --no-mcp skip the question.
Main package	@click-to-source-3d/vite-plugin from 0.1.5. overlay becomes legacy.
Hand-written tags	Never required. Kept as an optional override.
Versions	0.1.x stays React Three Fiber. 0.2.0 is the first version supporting both Three.js and R3F.

PART 10

Risks and how they are handled

Risk	Handling
R3F's list of canvases is a semi-private API and could change.	Checked at runtime, with a fallback that redraws by re-issuing the last frame.
The official three.js devtools browser extension uses the same hook.	Attach to the existing hook instead of replacing it, so both work.
Vite might re-bundle dependencies and reload the page on first run.	Configure it up front, and verify on a fresh pnpm app before release.
Alt+Shift+C may clash with an app's own shortcut.	Configurable, and ignored while typing in text fields.
Plain Three.js edge cases (Part 6.3).	Each falls back to showing the creation line, never to wrong information.
Instrumentation slowing apps down.	Only generator functions are wrapped; measured no difference in the prototype.

GLOSSARY

Terms used in this document

Term	Meaning
Three.js	The JavaScript library most web 3D is built with.
React Three Fiber (R3F)	A way to write Three.js scenes as React components, in JSX.
Vite	The dev server and build tool most R3F and Three.js projects use. <code>npm run dev</code> usually starts it.
Vite plugin	An add-on that changes how Vite builds and serves your code. Ours tags meshes and adds the inspector.
Hot reload (HMR)	Vite updating the page instantly when a file changes, without a full refresh.
Tag / stamp	The file, function and line attached to each object at build time.
Provenance	Where something came from: the file, line and call chain that produced it.
InstancedMesh	One mesh drawing many copies of the same shape (like a forest of trees) for speed.
MCP	Model Context Protocol: the open standard that lets AI assistants use tools from other programs (Part 4).
MCP server	A program that offers tools to AI assistants over MCP. Ours is <code>click-to-source-mcp</code> .
Bridge	The connection the dev server uses to ask your open page about the live scene.
Monorepo	One repository holding several packages, such as an app and a shared UI library.